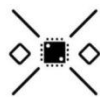




智能服务机器人 使用指导书



塔洛斯实验室
TALOS

北方工业大学TALOS机器人团队（编著）

目 录

任务一：linux 基础	4
任务二：启动机器人.....	7
任务三：主从机配置.....	10
任务四：VNC 远程桌面连接	14
任务五：键盘控制.....	15
任务六：启动深度相机采集 RGB 图像和深度图像	17
任务七：语音交互.....	23
任务八：启动雷达.....	26
任务九：加载 gazebo 仿真模型.....	29
任务十：地图构建.....	31
任务十一：自主导航.....	36
任务十二：多点导航.....	39
任务十三：传感器控制	44
任务十四：综合任务.....	48

ROS 简介

ROS，英文全称 Robot Operating System，即机器人操作系统，它提供了操作系统应有的服务，包括硬件抽象，底层设备控制，常用函数的实现，进程间消息传递，以及包管理。它也提供用于获取、编译、编写、和跨计算机运行代码所需的工具和库函数。

ROS 作为一个开源的软件系统，在某些方面 ROS 相当于一种“机器人框架 (robot frameworks)” 宗旨是构建一个能够整合不同研究成果，实现算法发布、代码重用、的通用机器人软件平台，其中包含一些列的工具、库和约定。同时，ROS 还可以为异质计算机集群，提供了类似操作系统的中间件。并且很多开源的运动规划、定位导航、仿真、感知等软件功能包使得这一平台的功能变得更加丰富，发展更加迅速。到目前为止，ROS 在机器人的感知、物体识别、脸部识别、姿势识别、运动、运动理解、结构与运动、立体视觉、控制、规划等多个领域都有相关应用。

ROS 将每个工作进程都看作是一个节点，使用节点管理器进行统一管理。并提供了一套消息传递机制。可以分散由计算机视觉和语音识别等功能带来的实时计算压力，能够适应多机器人遇到的挑战。

过程 总览	任务一：linux 基础	1. 学习 cd 命令 2. 学习 ls 命令 3. 学习 pwd 命令 4. 学习 gedit/nano 编辑 5. 学习 cp 命令
	任务二：启动机器人	1. 打开机器人各个开关。 2. 显示器连接方法
	任务三：主从机配置	1. 主从机配置 2. ssh远程连接
	任务四：vnc远程连接	1. vnc远程连接
	任务五：键盘控制	1. 控制小车移动
	任务六：打开相机	1. 打开相机 2. 采集图像 3. 老人摔倒检测 4. 人脸识别
	任务七：语音交互	1. 语音唤醒 2. 语音识别
	任务八：启动雷达	1. 驱动参数说明 2. 滤波参数说明
	任务九：加载gazebo模型	1. 加载机器人模型
	任务十：地图构建	1. 通过 gmapping 或Cartographer构建地图 2. 保存地图
	任务十一：自主导航	1. 单点导航
	任务十二：多点导航	1. 多点导航
	任务十三：传感器控制	1. 传感器控制
	任务十四：综合任务	1.家居场景功能实现

任务一：linux 基础

安装 ubuntu16.04 系统之后，为了便于操作，以下列出 linux 的常用基本操作命令，这些操作将会贯穿整个智能车任务中，请务必多加练习以掌握 linux 基础操作。

cd	切换目录; (Switching directories)
ls	列出目录; (List directories)
pwd	显示目前的目录; (Displaysthecurrentdirectory)
gedit/nano	文档编辑器; (Document Editor)
cp	复制文件或目录; (Copy a file or directory)

1、linux 基本命令:

cd:切换目录，这是用来变换工作目录的命令。

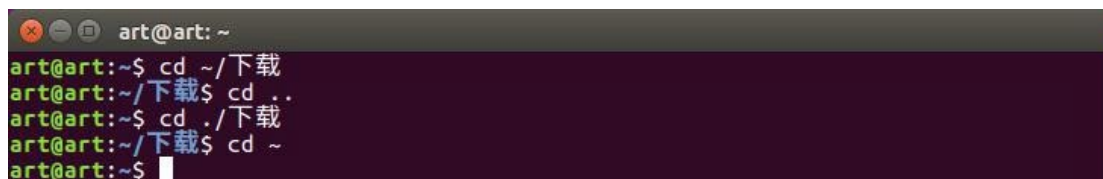
输入 **cd ~/下载** 回车切换工作目录到“下载”文件夹下: 输

入 **cd ..** 回车返回上一级目录:

其中“.”是指在当前目录下，“..”是指上一级目录，“~”是指 home 目录

输入 **cd ./下载** 回车切换工作目录到“下载”文件夹下。输

入 **cd ~** 回车返回 home 目录:



```
art@art: ~  
art@art:~$ cd ~/下载  
art@art:~/下载$ cd ..  
art@art:~$ cd ./下载  
art@art:~/下载$ cd ~  
art@art:~$
```

使用 **pwd** 命令可以显示当前终端运行所在目录。使用 **ls** 命令可显示当前目录下的所有文件。

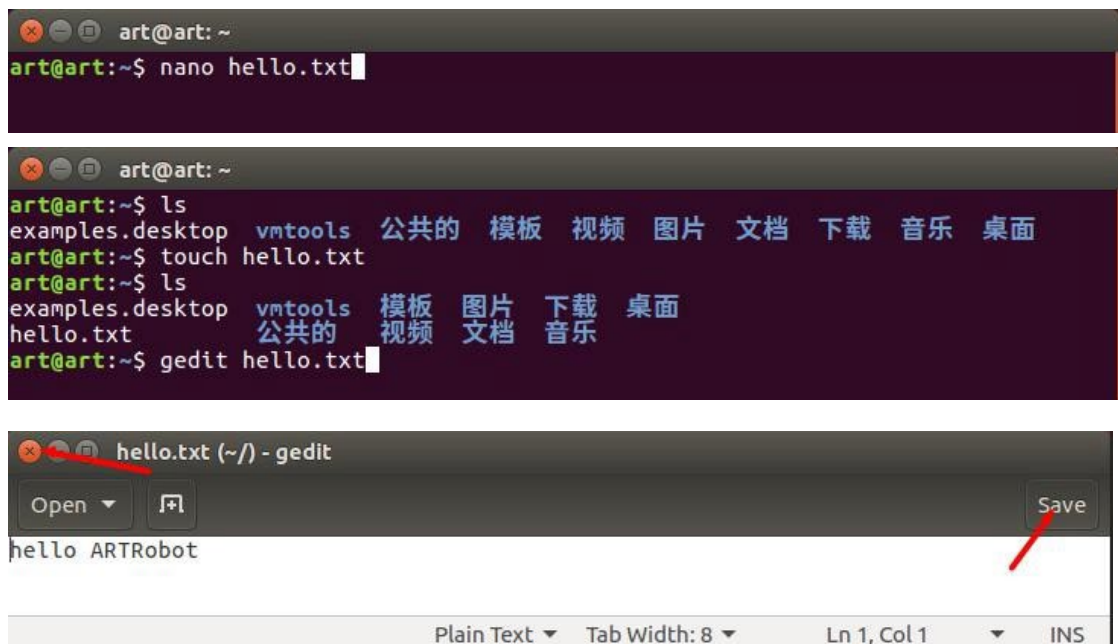
```
art@art: ~  
art@art:~$ pwd  
/home/art  
art@art:~$ ls  
examples.desktop  vntools  公共的  模板  视频  图片  文档  下载  音乐  桌面  
art@art:~$
```

从图中可以看出，现在终端的运行目录在/home/art

2.学习使用 gedit 编辑器和 nano 编辑器:

首先打开一个终端，在主目录下创建一个 hello.txt 文件，并利用 gedit 和 nano 对其进行编译。使用 ls 命令列出当前目录下所包含的内容。在终端中利用 touch hello.txt 回车创建一个 hello.txt 文件，并在上述终端中输入 gedit hello.txt 回车，打开该文件。文件中没有内容，输入“hello ARTRobot”并单击“保存”后关闭。

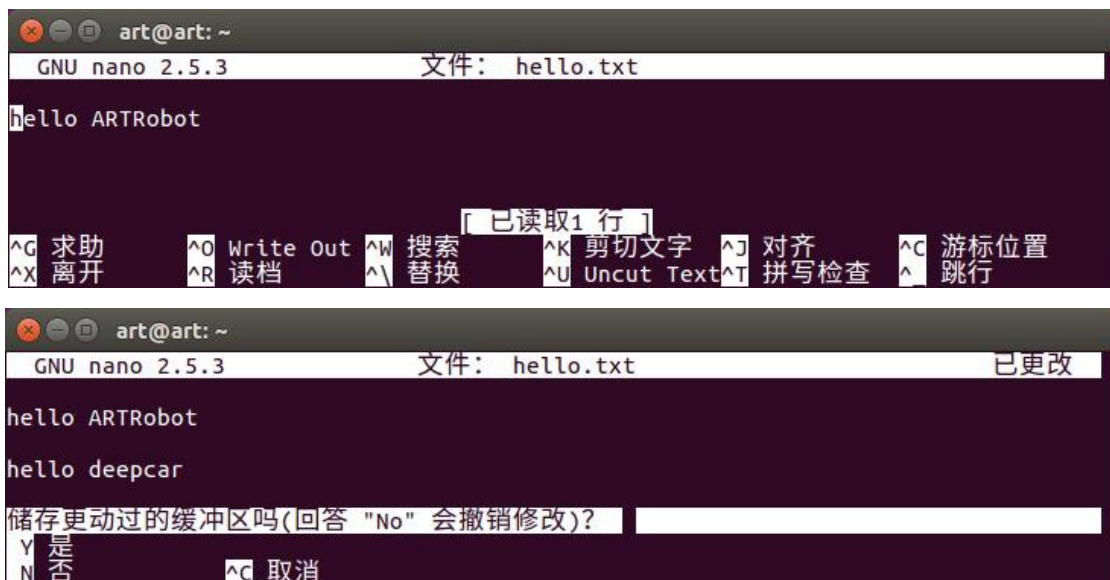
另一种方法,用 nano 编辑器打开文档。在终端中输入 nano + 文件名,打开文档。



回车之后,可以在终端中打开了文档。

可以看到刚才写入的”hello ARTRobot”,接着在这个编辑器中上下左右移动光标位置,将光标移至”hello ARTRobot”后方回车,输

入“hello deepcar”.然后按 `ctrl+x` 离开该界面，输入 `y` 回车保存修改。



```
art@art: ~  
GNU nano 2.5.3 文件: hello.txt  
hello ARTRobot  
[已读取1行]  
^G 求助      ^O Write Out  ^W 搜索      ^K 剪切文字  ^J 对齐      ^C 光标位置  
^X 离开      ^R 读档      ^\ 替换      ^U Uncut Text ^T 拼写检查  ^_ 跳行  
  
art@art: ~  
GNU nano 2.5.3 文件: hello.txt 已更改  
hello ARTRobot  
hello deepcar  
储存更动过的缓冲区吗(回答 "No" 会撤销修改)?  
Y 是  
N 否      ^C 取消
```

`cp` 命令可以复制替换文件或目录。使用指令 `cp file1 file2` 是将文件 `file1` 复制为 `file2`。可以用刚才的 `hello.txt` 演示。在终端输入 `cp hello.txt HELLO.txt` 并回车,然后使用 `ls` 指令,可以看出该目录下,多了个 `HELLO.txt` 文件。



```
art@art: ~  
art@art:~$ ls  
examples.desktop  vmtools  模板  图片  下载  桌面  
hello.txt         公共的  视频  文档  音乐  
art@art:~$ cp hello.txt HELLO.txt  
art@art:~$ ls  
examples.desktop  HELLO.txt  公共的  视频  文档  音乐  
hello.txt         vmtools   模板  图片  下载  桌面  
art@art:~$
```

任务二：启动机器人

本节我们将介绍机器人所用到的设备，以及如何启动机器人控制界面和系统的密码。

1、准备设备：

Ros 助老服务机器人：一台

显示屏：一台

HDMI 接口线：一根

USB 线：一根

无线鼠键：一套

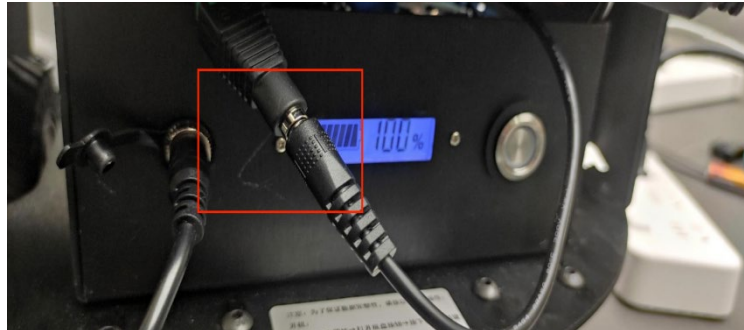
2、显示器连接方法：

显示屏的连接方法：HDMI 接口线接在工控机的HDMI 接口，USB 线接在小车工控机上面USB 接口上。然后既可开始启动机器人。



3、机器人启动

1) 打开机器人底盘：连接电池电源线（无外露电源线的忽略此步）：



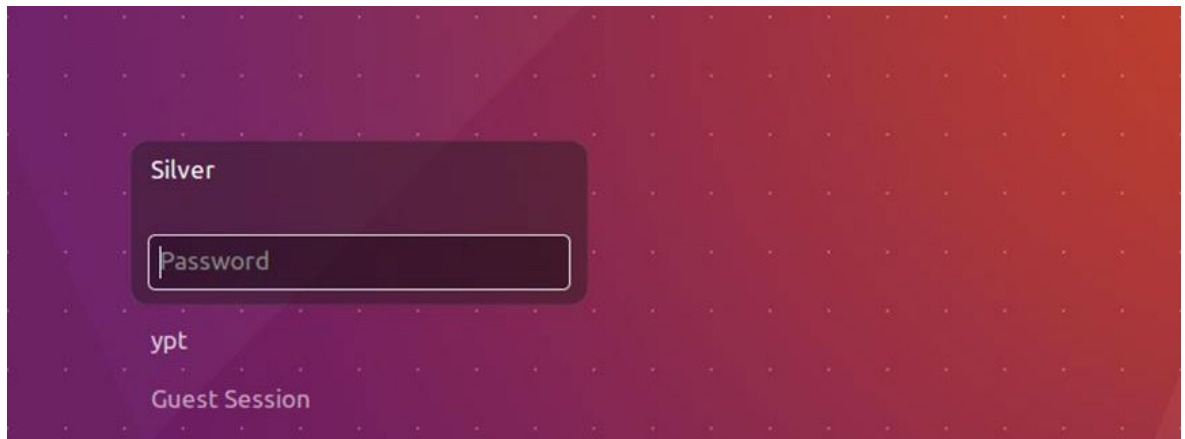
按下下图的按钮开机：



2) 打开工控机：



3) 等待机器人开机，显示屏出现如下画面。进入登陆页面，默认密码为：**a**，输入密码后回车。



4) 此时我们就可以开始对机器人的操作了。

任务三：主从机配置

1.使用鼠标右键打开终端或者同时按住 **ctrl+alt+t** 打开终端，输入 **ifconfig** 查看工控机的ip，再输入**hostname**查看工控机的主机名：

```
ub@ub:~$ ifconfig
enp1s0    Link encap:Ethernet  HWaddr 00:e0:0a:61:23:3f
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:16026 errors:0 dropped:0 overruns:0 frame:0
          TX packets:16026 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:885588 (885.5 KB)  TX bytes:885588 (885.5 KB)

wlp2s0    Link encap:Ethernet  HWaddr 70:18:8b:a2:0b:ef
          inet addr:10.42.0.1   Bcast:10.42.0.255  Mask:255.255.255.0
          inet6 addr: fe80::7218:8bff:fea2:bef/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:19806 errors:0 dropped:2 overruns:0 frame:0
          TX packets:33173 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1177298 (1.1 MB)  TX bytes:47965051 (47.9 MB)

ub@ub:~$ hostname
ub
ub@ub:~$
```

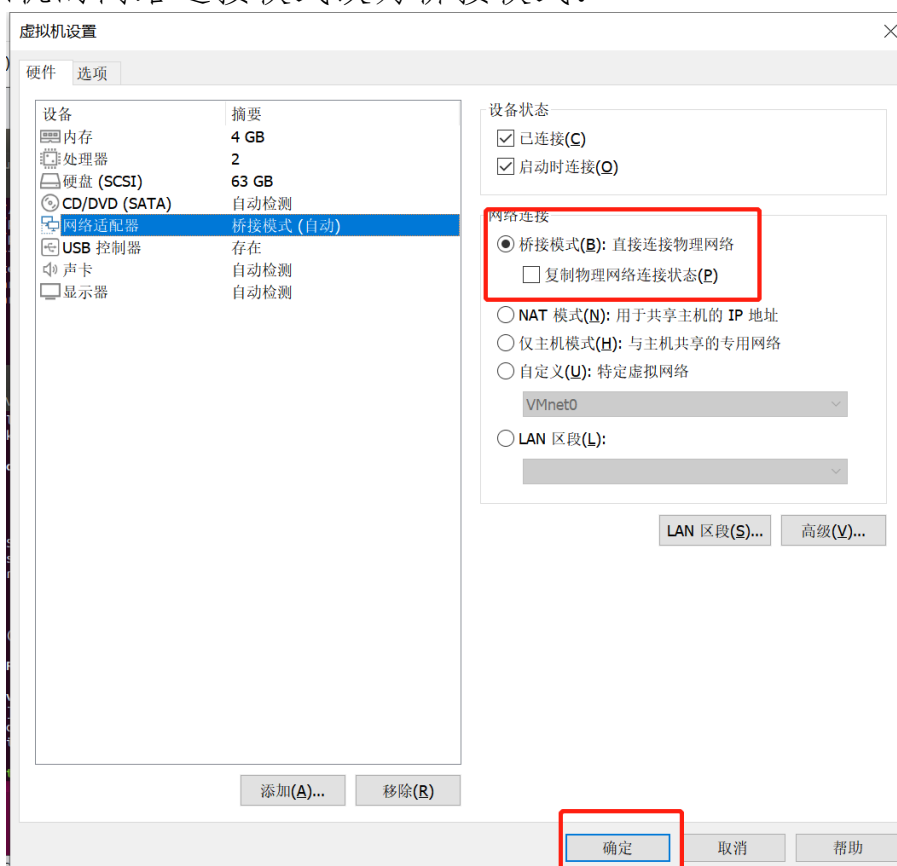
ub为用户名，ub 为主机名，~为表示当前在 home 目录下。

\$表示目前以普通用户身份运行命令，#则表示以超级管理员权限运行。

2.widows主机连接工控机的wifi,wifi名字为ub，密码无：



Ubuntu虚拟机的网络连接模式改为桥接模式：



重启ubuntu网络，查看Ubuntu的ip地址和hostname：

```
ncut@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.42.0.221 netmask 255.255.255.0 broadcast 10.42.0.255
    ether 00:0c:29:58:48:de txqueuelen 1000 (Ethernet)
    RX packets 141226 bytes 106744625 (106.7 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 95859 bytes 6569532 (6.5 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 27121 bytes 1933088 (1.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 27121 bytes 1933088 (1.9 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ncut@ubuntu:~$ hostname
ubuntu
ncut@ubuntu:~$
```

3.在工控机路径/etc/下打开hosts文件加入ubuntu的ip和hostname： 如图：

```

127.0.0.1    localhost
127.0.1.1    ub
10.42.0.1    ub
# The following lines are desirable for IPv6 capable hosts
::1         ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
10.42.0.221 ubuntu

```

4.在ubuntu路径/etc/下打开hosts文件，加入工控机的ip和hostname:

```

Open ▾  hosts
/etc/

127.0.0.1    localhost
127.0.1.1    ubuntu

# The following lines are desirable for IPv6 capable hosts
::1         ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters

151.101.84.133 raw.githubusercontent.com
192.168.12.1  EPRobot
192.168.12.221 ubuntu
192.168.8.125 ucar-mini
10.42.0.221| ubuntu
10.42.0.1    ub

```

在home目录下打开bashrc加入主机ip和从机hostname:

```

source /home/ncut/gazebo_test_ws/devel/setup.bash
export LIBGL_ALWAYS_SOFTWARE=1

export ROS_MASTER_URI=http://10.42.0.1:11311
export ROS_HOSTNAME=ubuntu

#when use matlab, set this ip 10.42.0.1
#export ROS_IP=192.168.31.43

```

完成后打开终端输入ping 10.42.0.1,若出现以下数据则说明主从机配置成功:

```

ncut@ubuntu:~$ ping 10.42.0.1
PING 10.42.0.1 (10.42.0.1) 56(84) bytes of data.
64 bytes from 10.42.0.1: icmp_seq=1 ttl=64 time=12.2 ms
64 bytes from 10.42.0.1: icmp_seq=2 ttl=64 time=5.13 ms
64 bytes from 10.42.0.1: icmp_seq=3 ttl=64 time=16.1 ms
64 bytes from 10.42.0.1: icmp_seq=4 ttl=64 time=32.3 ms
64 bytes from 10.42.0.1: icmp_seq=5 ttl=64 time=15.1 ms
64 bytes from 10.42.0.1: icmp_seq=6 ttl=64 time=13.0 ms
64 bytes from 10.42.0.1: icmp_seq=7 ttl=64 time=25.8 ms
64 bytes from 10.42.0.1: icmp_seq=8 ttl=64 time=21.7 ms
64 bytes from 10.42.0.1: icmp_seq=9 ttl=64 time=12.7 ms
^[[a64 bytes from 10.42.0.1: icmp_seq=10 ttl=64 time=13.1 ms
64 bytes from 10.42.0.1: icmp_seq=11 ttl=64 time=19.9 ms
64 bytes from 10.42.0.1: icmp_seq=12 ttl=64 time=17.2 ms

```


任务四：VNC远程桌面连接

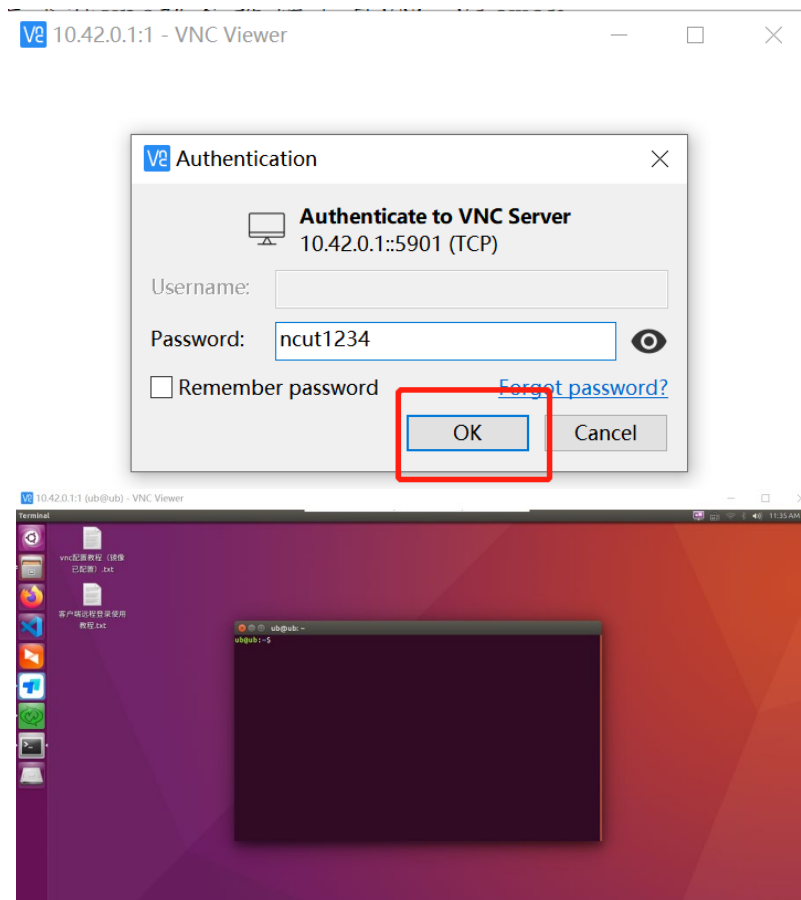
1. 在windows上下载vnc远程连接工具VNC Viewer



2. 打开VNC Viewer，输入工控机的ip加端口号（默认端口号是1）



输入vnc默认密码ncut1234,进入远程连接界面：



任务五：键盘控制

机器人启动之后，就可以开始让机器人动起来，本节我们将使用按键控制功能实现机器人的移动。

- 1、机器人启动完毕后，按 **Ctrl+Alt+T** 或者右键点击Terminal图标，弹出终端窗口□。
- 2、在终端□输入启动命令，并回车：

roslaunch base_control stm32.launch

```
ub@ub:~$ roslaunch base_control stm32.launch
... logging to /home/ub/.ros/log/17f9af04-e7d2-11ed-8183-70188ba20bef/roslaunch-ub-32042.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ub:33531/

SUMMARY
=====
PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.17
* /stm32_node/base_id: base_footprint
* /stm32_node/base_kd: 5.0
* /stm32_node/base_ki: 17.0
* /stm32_node/base_kp: 0.5
* /stm32_node/base_kv: 1.0
* /stm32_node/baudrate: 115200
* /stm32_node/cam_table_topic_: /cam_movie
* /stm32_node/cmd_vel_topic: /cmd_vel
* /stm32_node/is_pub_odom_tf: False
```

- 3、再次打开新的命令窗□，启动键盘控制节点，输入以下指令：

roslaunch teleop_twist_keyboard teleop_twist_keyboard.py

```
ub@ub:~$ roslaunch teleop_twist_keyboard teleop_twist_keyboard.py
Reading from the keyboard and Publishing to Twist!
-----
Moving around:
   u   i   o
   j   k   l
   m   ,   .

For Holonomic mode (strafing), hold down the shift key:
-----
```

4、此时即可通过控制键盘上的 **u**、**i**、**o**、**j**、**k**、**l**、**m**、**U**、**I**、**O**、**J**、**K**、**L**、**M** 等按键去控制机器人的移动。

注意：执行完成每小节操作后，务必通过 **Ctrl+C** 终结节点，然后关闭终端，随后再去练习下一个小节的内容，否则各个节点之间可能会造成冲突，影响机器人的使用。

任务六：启动深度相机采集RGB图像和深度图像

待机器人完成基本的移动功能之后，就可以加入更多的传感器，配备一个深度相机，其中可以获取小车前方的深度信息和 RGB 图像信息。我们可以使用深度信息实现基于 3D 视觉的 SLAM 建图功能，也可以使用 RGB 图像完成图像处理的一系列功能。丰富机器人的功能。本节介绍如何打开深度相机和获取图像以及对老人摔倒检测和人脸识别：



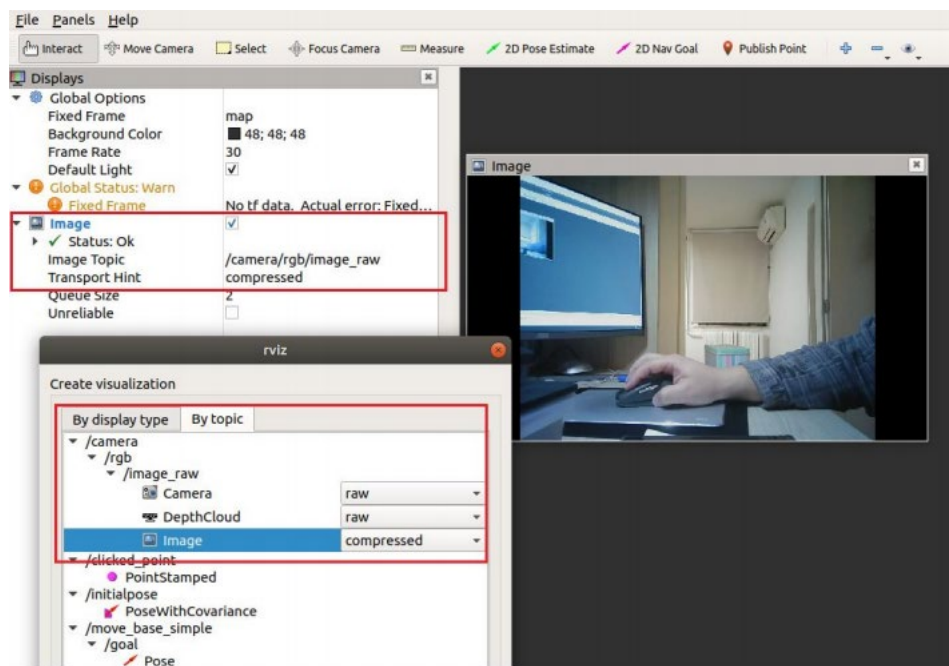
1、首先我们需要关闭上一节打开的终端，然后开一个新终端：

```
roslaunch astra_launch astra.launch
```

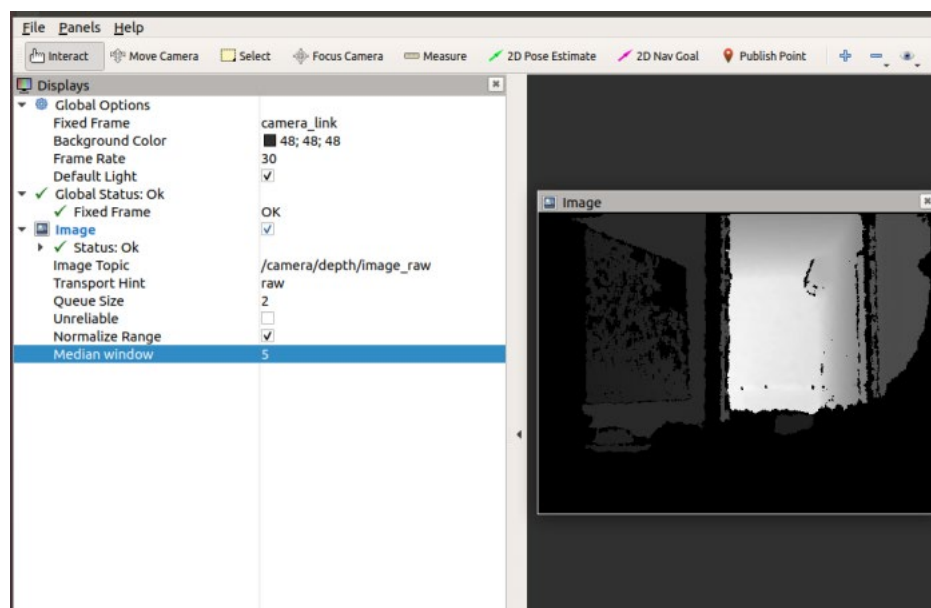
2、启动rviz:

```
rviz
```

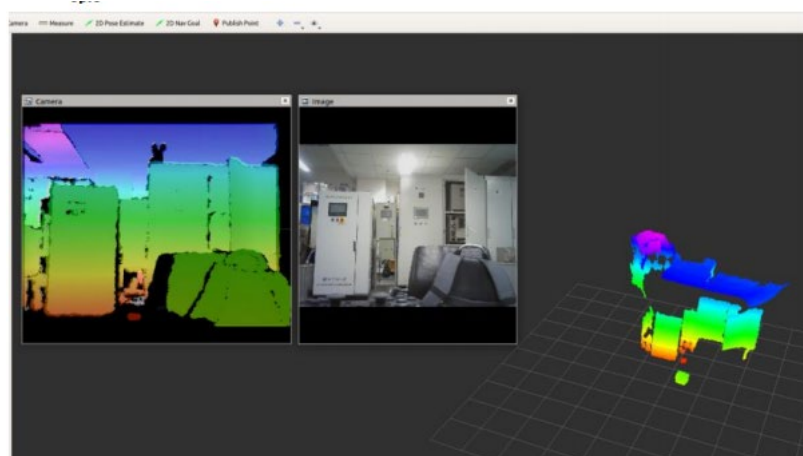
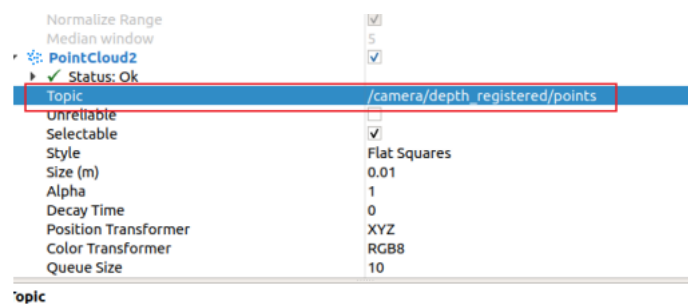
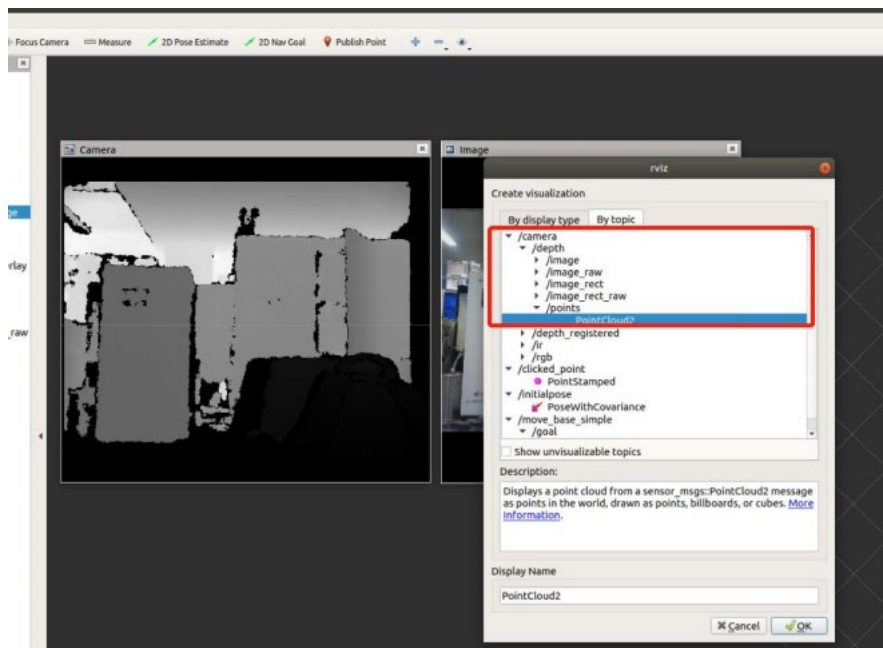
3.在界面中新建可视化，选择“by topic”中的 rgb image 话题，如下图所示，即可看到 rgb 图像输出。



4.在界面中新建可视化，选择“by topic”中的 depth camera 话题，如下图所示，即可看到深度图像输出。



5.在界面中新建可视化，选择“by topic”中的 depth pointsCloud2 话题，将global option 中的 Fixed Frame 设置为camera_link，并设置 pointsCloud2 的话题为/camera/depth_registered/points，如下图所示，即可看到深度点云输出。



可以尝试切换 `pointsCloud2` 中的 `Color Transformer`，来修改点云的显示效果。

任务六：启动深度相机进行老人摔倒检测

摔倒作为老人常见的意外事故之一，往往导致严重后果。为了及时有效地检测老人摔倒事件，我们提出了基于ROS（Robot Operating System）的深度相机与YOLO（You Only Look Once）算法相结合的摔倒检测系统。

具体操作步骤如下：

- 1.启动方法:直接在本地终端或者远程连接工控机主机wifi:ub2，打开远程终端，输入上一节打开深度相机的指令：

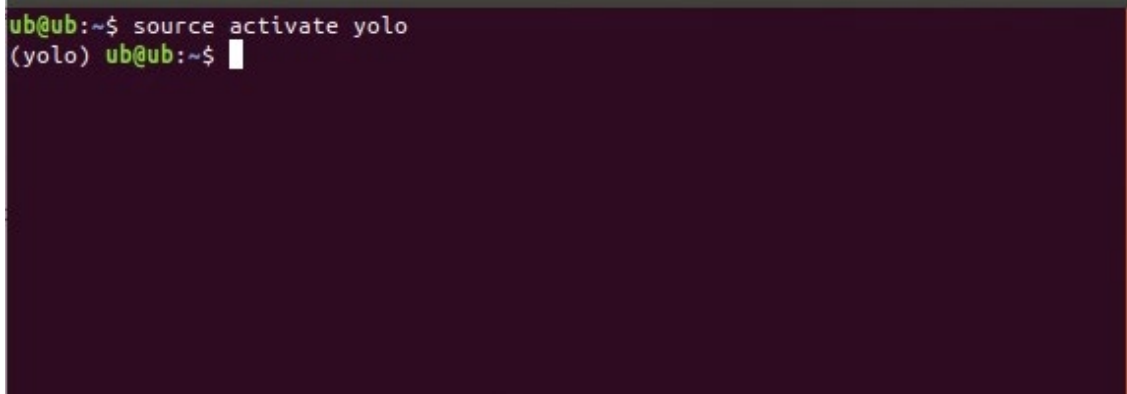
roslaunch astra_launch astra.launch

- 2.打开一个新终端，使用web_video_server进行网页段的视频传输，输入指令：

roslaunch web_video_server web_video_server

- 3.再次打开一个新终端，进入yolo的虚拟环境，输入指令：

source activate yolo



```
ub@ub:~$ source activate yolo
(yolo) ub@ub:~$
```

- 4.在上个终端下，进入老人摔倒检测的文件路径下，输入指令：

cd /home/ub/yolov5-mask-42-master/

- 5.在上个终端下，运行老人摔倒的程序，将会弹出一个视频框，并将摔倒的人框出来，并进行‘检测到老人摔倒’的语音播报，输入指令：

python3 detect1.py

任务六：启动深度相机进行人脸识别

我们结合了ROS的强大分布式处理能力和OpenCV在计算机视觉领域的丰富功能，开发了一套基于ROS与OpenCV的人脸识别系统,但是此人脸识别置信度不是特别高，还需后续自己完善。

具体操作步骤如下：

- 1.启动方法:直接在本地终端或者远程连接工控机主机wifi:ub2，打开远程终端，输入上一节打开深度相机的指令：

```
roslaunch astra_launch astra.launch
```

- 2.打开一个新终端，进入人脸识别的文件路径下：

```
cd /home/ub/omniWheelCareRobot/rosCode/src/face_recognition/script
```

- 3.对该路径下的face_dataset.py进行修改，如下图所示，

self.sample_face_max是指采集的一个的人脸图片的数目，self.face_id是指采集人脸的序号，对人脸进行采集，在上面的终端下输入：

```
python2 face_dataset.py
```

```
# Initialize individual sampling face count  
self.sample_face_max = rospy.get_param('~sample_face_max', 15)  
  
self.face_id = rospy.get_param('~face_id', '2')
```

- 4.待把所有的人脸采集完成后，对人脸进行训练，在上面的终端输入：

```
python2 face_training.py
```

- 5.待把人脸训练完成后，修改文件face_recognition.py，如下图所示对人脸id进行修改，然后对人脸进行识别，在上面的终端输入：

```
python2 face_recognition.py
```

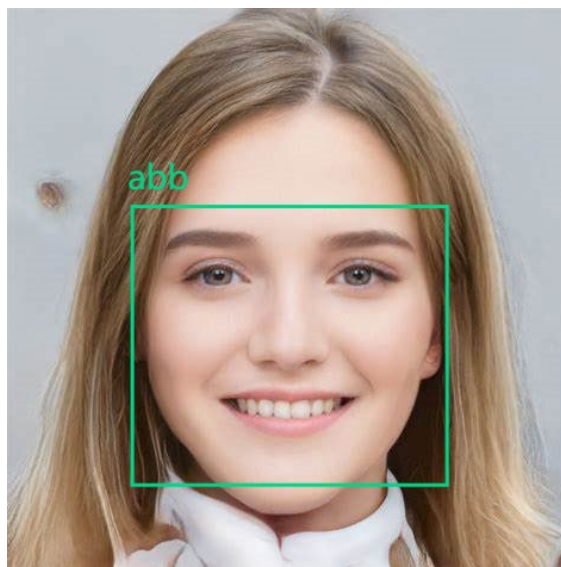
6.待把人脸训练完成后，修改文件face_recognition.py，如下图所示对人脸id进行修改，然后对人脸进行识别，识别人脸的序号和置信度将在终端输出相关信息，在上面的终端输入：

python2 face_recognition.py

```
# names related to ids: example ==> Marcelo: id=1, etc
#names = ['None','id1','id2','id3','id4','id5','id6']
names = ['None','id1','id2']
```

7.如果想把人脸识别可视化，打开rviz，如下图所示，选择“by topic”中的/image_face话题，识别到人脸用绿色边框框住人脸，并且在方框的右上角的显示人脸的id，启动rviz，需要打开一个新终端，输入：

rviz



任务七：语音交互

语音交互是人与机器人沟通的一种常用方式。助老服务机器人适配了科大讯飞的麦克风阵列以及语音播报驱动程序，可以实现语音识别、声源定位、语音播报以及语音控制功能。

具体操作步骤如下：

1.启动方法:直接在本地终端或者远程连接工控机主机wifi:ub2，打开远程终端，输入：

roslaunch xf_mic_asr_offline voice_init.launch

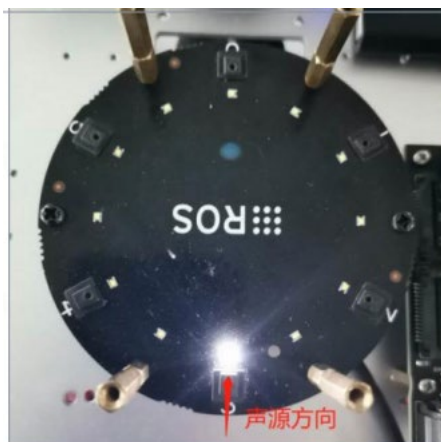
此时，终端会输出以下信息，如下图：

```
[voice_control] source_path=/home/EPRobot/robot_ws/src/xf_mic_asr_offline
[voice_control] appid=87b9d5d7
[voice_control] -->>>>成功打开麦克风设备
[voice_control] -->>>>麦克风正在启动,软件版本为:R-2.2,协议版本为:
[cmd_rec] 您可以语音控制啦！唤醒词“小易同学”
[cmd_rec] 小车前进————>向前
[cmd_rec] 小车后退————>后退
[cmd_rec] 小车左转————>左转
[cmd_rec] 小车右转————>右转
[cmd_rec] 小车停————>停止
[cmd_rec] 小车休眠————>休眠，等待下一次唤醒
[cmd_rec] 小车过来————>寻找声源
[cmd_rec] 小车去I点————>小车自主导航至I点
[cmd_rec] 小车去J点————>小车自主导航至J点
[cmd_rec] 小车去K点————>小车自主导航至K点

[voice_control] -->>>>开机中，请稍等！
[INFO] [1624978926.936826]: base control start...
[INFO] [1624978926.982118]: Opening Serial
[INFO] [1624978926.985713]: Serial Open Succeed
[voice_control] -->>>>开机成功！
唤醒词是:小易同学;nCM:600
[voice_control] -->>>>未设置主麦，请唤醒或设置主麦
```

根据提示，我们可知语音模块已经开机成功。机器人的唤醒词是“**小微**”。当你对着小车读出唤醒词后，如果语音模块正常识别，会做以下工作：

(1) 识别声源的方位，并点亮相应的LED灯，如下图所示。通过LED灯可以判断是否识别准确，也可以通过终端输出信息确认；



```
唤醒词是:小易同学;nCM:600
[voice_control] -->>>>未设置主麦，请唤醒或设置主麦
[voice_control] -->>>>第2个麦克风被唤醒
[voice_control] -->>>>唤醒角度为:103
[voice_control] -->>>>已点亮3灯
小车唤醒
[cmd_rec] 小车已被唤醒，请说语音指令
>>>>开始录制降噪音频
[voice_control] -->>>>开始一次语音识别！
[voice_control] -->>>>设置主麦成功！
```

(2) 将所识别到的声源方位的麦克风设置为主麦，其他麦克风设置为噪音收集麦克，以达到更好的降噪效果；

(3) 若小车安装了扬声器，小车会进行语音播报，以提示唤醒成功；

```
string str2 = "小车前进";  
string str3 = "小车后退";  
string str4 = "小车左转";  
string str5 = "小车右转";  
string str6 = "小车停止";  
string str7 = "小车休眠";  
string str8 = "小车过来";  
string str9 = "小车去I点";  
string str10 = "小车去J点";  
string str11 = "小车去K点";  
string str12 = "失败5次";  
string str13 = "失败10次";  
string str14 = "遇到障碍物";  
string str15 = "小车唤醒";  
string str16 = "开始导航";  
string str17 = "小车几点";  
string str18 = "小车天气";  
string str19 = "小车讲解";
```

(4) 当唤醒小车后，便可按照启动时的终端提示进行语音控制。如上图所示目前小车只能够响应“小车前进”、“小车后退”、“小车停”、“小车左转”以及“小车右转”，“小车天气”，更多的响应可自行编程实现。需要注意的是，该麦克风阵列的语义识别是按照关键词进行的，小车搭载的功能包中，将“小车”设定为主语关键词，“后退”、“前进”为动词关键词，只要一句话中包含这两组关键词，小车就可以响应。更多的功能可参考网盘中提供的“硬件资料”文件夹中，讯飞麦克风阵列相关开发资料，以及讯飞官网的信息。

离线命令词识别功能和SDK介绍：

<https://www.xfyun.cn/doc/asr/commandWord/Linux-SDK.html>

API接口：

<https://www.xfyun.cn/doc/mscapi/Windows&Linux/WLfilelist.html>

错误码查询：

<https://www.xfyun.cn/document/error-code>

任务八：启动雷达

助老服务机器人配备的激光雷达主要用于扫描周围的障碍物，以及时避让，在建图和导航过程中具有很重要的作用，启动雷达节点具体操作步骤如下：

1、首先我们需要关闭上一节打开的终端，然后开一个新终端，构建地图时，我们需要启动机器人，并开启 雷达功能包节点，输入如下指令：

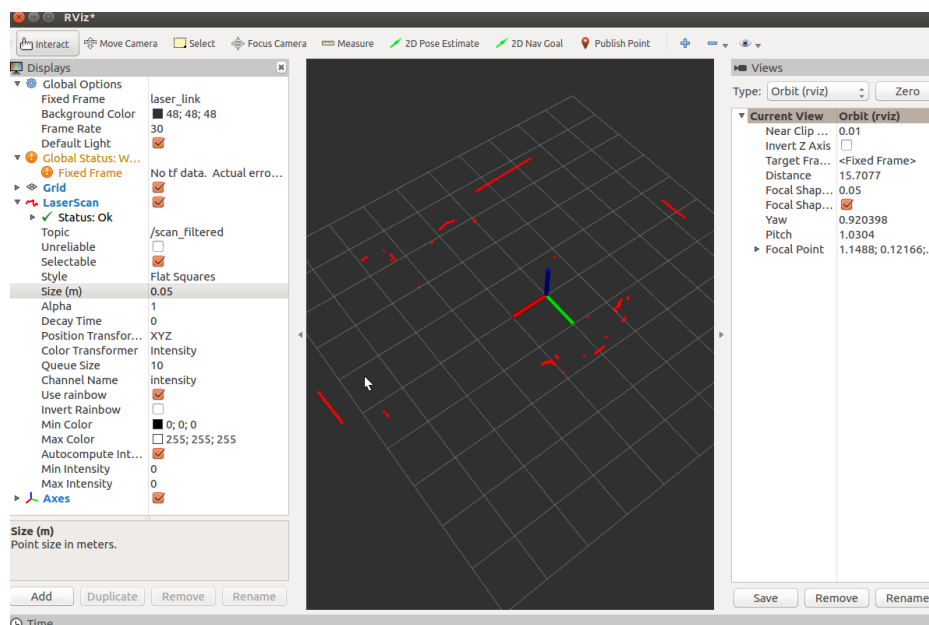
roslaunch carebot_navigation laser_filter.launch

```
/home/ub/omniWheelCareRobot/rosCode/src/carebot_navigation/launch/laser_filter.launch
ub@ub:~$ roslaunch carebot_navigation laser_filter.launch
... logging to /home/ub/.ros/log/5dffff6e-eabe-11ed-91c1-c48e8fc20ed1/roslaunch-ub-12703.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ub:39713/

SUMMARY
=====
PARAMETERS
* /laser_filter/scan_filter_chain: [{'type': 'laser_...
* /lslidar_driver_node/angle_disable_max: 0
* /lslidar_driver_node/angle_disable_min: 0
* /lslidar_driver_node/compensation: False
* /lslidar_driver_node/frame_id: laser_link
* /lslidar_driver_node/high_reflection: False
* /lslidar_driver_node/interface_selection: serial
* /lslidar_driver_node/lidar_name: M10
* /lslidar_driver_node/max_range: 100.0
* /lslidar_driver_node/min_range: 0.0
* /lslidar_driver_node/pubPointCloud2: False
```

2、打开新终端，用来调用 **rviz** 去显示雷达数据，输入如下指令：**rviz** 并在**global option**中设置**fixed frame**为**laser_link**,并添加显示雷达的话题**Laser_scan**，就可以看到图中显示的障碍物：



3. 雷达的驱动参数位于lslidar_driver功能包下面的lslidar_serial.launch里面：

```
<launch>
<node pkg="lslidar_driver" type="lslidar_driver_node" name="lslidar_driver_node" output="screen">
  <param name="lidar_name" value="M10"/> #雷达选择:M10 M10_P M10_PLUS M10_GPS N10 L10 N10_P
  <param name="serial_port" value="/dev/carebot_laser_ls"/> #雷达连接的串口
  <param name="interface_selection" value="serial"/> #接口选择:net 为网口,serial 为串口。
  <!--param name="frame_id" value="laser_link"/--> #激光坐标
  <param name="frame_id" value="laser_link"/>
  <param name="scan_topic" value="scan"/> #设置激光数据topic名称
  <param name="angle_disable_min" value="0"/> #角度裁剪开始值
  <param name="angle_disable_max" value="0"/> #角度裁剪结束值
  <param name="min_range" value="0.0"/> #雷达接收距离最小值
  <param name="max_range" value="100.0"/> #雷达接收距离最大值
  <param name="use_gps_ts" value="false"/> #雷达是否使用GPS授时
  <param name="compensation" value="false"/> #雷达是否使用角度补偿功能
  <param name="pubScan" value="true"/> #雷达是否发布scan话题
  <param name="pubPointCloud2" value="false"/> #雷达是否发布pointcloud2话题
  <param name="high_reflection" value="false"/> #M10_P雷达需填写该值,若不确定,请联系技术支持。
  <!--param name="in_file_name" value="$(find lslidar_driver)/pcap/1.txt"/--> #驱动是否使用txt文件读取功能
</node>
  <!--node name="rviz" pkg="rviz" type="rviz" args="-d $(find lslidar_driver)/rviz/lslidar.rviz" output="screen"/--> #雷达点云配置
</launch>
```

4.关于雷达滤波参数的配置，集中在laser_filters功能包路径下的examples配置文件，机器人雷达采用shadow_filter_example.yaml配置，如下图所示：

```

scan_filter_chain:
- name: shadows
  type: laser_filters/ScanShadowsFilter
  params:
    min_angle: 5
    max_angle: 175
    neighbors: 20
    window: 6
- name: box_filter
  type: laser_filters/LaserScanBoxFilter
  params:
    box_frame: laser_link      #base_laser_link
    max_x: 0.2
    max_y: 0.2
    max_z: 0.2
    min_x: -0.2
    min_y: -0.2
    min_z: -0.2
    invert: false

#- name: dark_shadows
# type: laser_filters/LaserScanIntensityFilter|
# params:
#   lower_threshold: 100
#   upper_threshold: 10000
#   disp_histogram: 0

```

(机器人雷达滤波参数配置须根据用户需求，对于不同的滤波范围采用不同的参数配置)

任务九：加载gazebo仿真模型

助老服务机器人配置了 gazebo 仿真模型，可在建图导航过程中，更直观的对机器人的位姿和运动状态进行观察，也可以更好地对机器人各个关节的相对状态掌握，按下面步骤加载机器人仿真模型：

打开终端，输入：

```
roslaunch carebot_urdf display.launch
```

```
ub@ub:~$ roslaunch carebot_urdf display.launch
... logging to /home/ub/.ros/log/7027b1be-eac3-11ed-91c1-c48e8fc20ed1/roslaunch-ub-13581.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

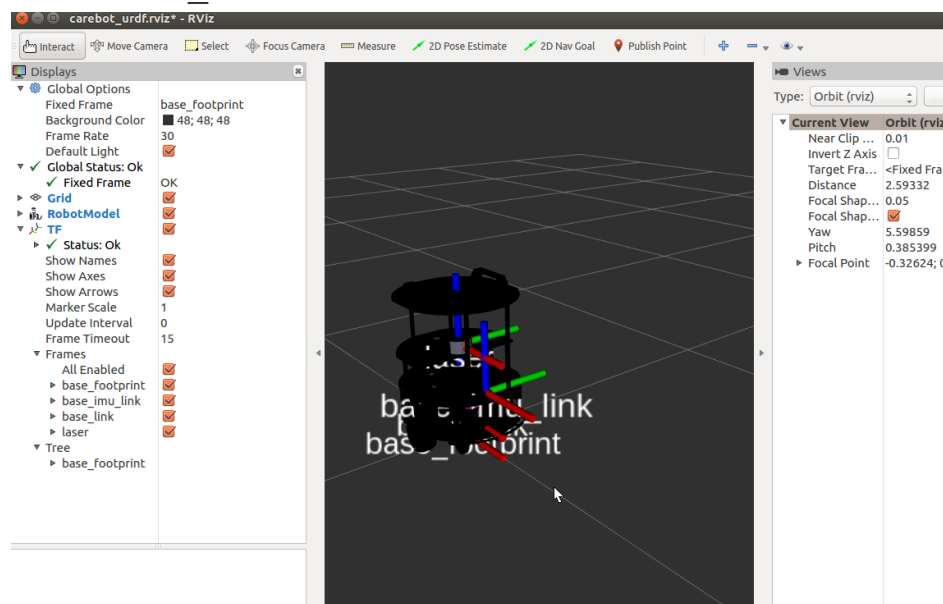
started roslaunch server http://ub:46155/

SUMMARY
=====

PARAMETERS
* /joint_state_publisher/rate: 10.0
* /robot_description: <?xml version="1...
* /robot_state_publisher/publish_frequency: 20.0
* /roscpp: kinetic
* /rosversion: 1.12.17
* /use_gui: False

NODES
/
```

在rviz中添加robot_model模型，显示如图所示：



如需将机器人模型运用到建图或导航中，可自行将本launch

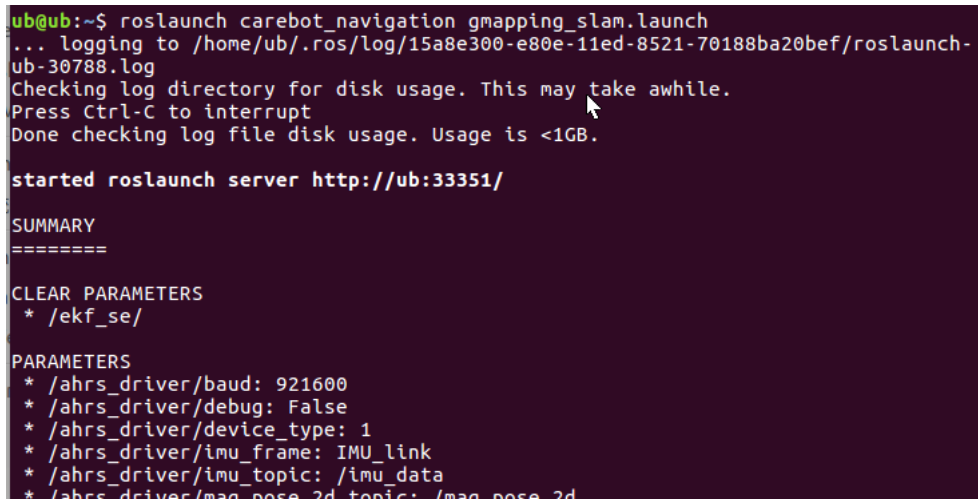
文件包含在建图或者导航启动launch文件中，方可在rviz中观察gazebo机器人模型。

任务十：地图构建

地图构建是实现机器人自主导航的必要条件，当我们构建好地图后机器人就可以依据地图信息实现导航效果。此处我们使用的是 **Gmapping** 功能包去构建地图，并通过 **map_server** 功能包去保存地图。

1、首先我们需要关闭上一节打开的终端，然后开一个新终端，构建地图时，我们需要启动机器人，并开启 **Gmapping** 功能包节点，输入如下指令：

roslaunch carebot_navigation gmapping_slam.launch



```
ub@ub:~$ roslaunch carebot_navigation gmapping_slam.launch
... logging to /home/ub/.ros/log/15a8e300-e80e-11ed-8521-70188ba20bef/roslaunch-ub-30788.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ub:33351/

SUMMARY
=====

CLEAR PARAMETERS
* /ekf_se/

PARAMETERS
* /ahrs_driver/baud: 921600
* /ahrs_driver/debug: False
* /ahrs_driver/device_type: 1
* /ahrs_driver/imu_frame: IMU_link
* /ahrs_driver/imu_topic: /imu_data
* /ahrs_driver/mag_pose_2d_topic: /mag_pose_2d
```

2、打开新终端，用来调用 **RViz** 去显示栅格地图，输入如下指令（如果地图已经打开，则跳过这个步骤）：

roslaunch carebot_navigation view_navigation.launch

```
/home/art/omniWheelCareRobot/rosCode/src/carebot_navigation/launch/view_navigation
art@art:~$ roslaunch carebot_navigation view_navigation.launch
... logging to /home/art/.ros/log/4ec83172-be64-11e9-976e-68a3c4584893/roslaunch
-art-17631.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://art:38523/

SUMMARY
=====

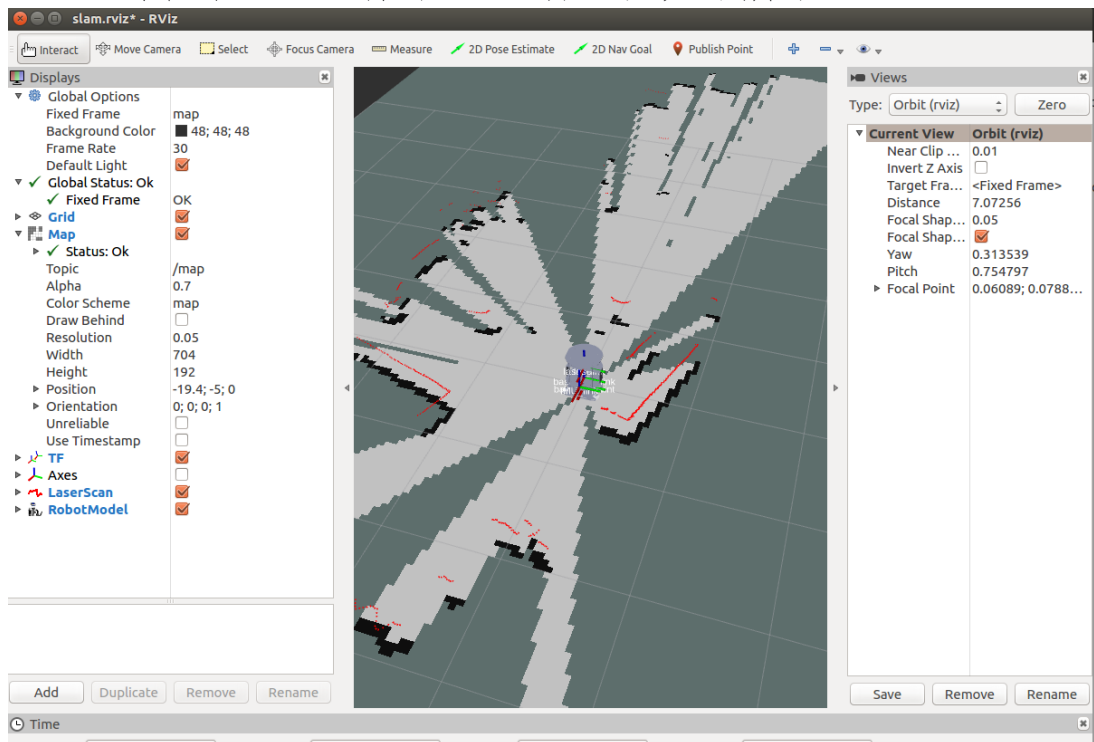
PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.13

NODES
/
  rviz (rviz/rviz)

ROS_MASTER_URI=http://localhost:11311

process[rviz-1]: started with pid [17648]
```

3、此时在桌面上会打开 RViz 界面，如下所示：



4、打开地图后，需要通过按键控制移动机器人，在空间中移动，扫描一个完整的地图，为随后导航使用。此时打开新终端，输入按键控制的指令：

roslaunch teleop_twist_keyboard teleop_twist_keyboard.py

```
art@art: ~
art@art:~$ rosrn teleop_twist_keyboard teleop_twist_keyboard.py

Reading from the keyboard and Publishing to Twist!
-----
Moving around:
  u    i    o
  j    k    l
  m    ,    .

For Holonomic mode (strafing), hold down the shift key:
-----
  U    I    O
  J    K    L
  M    <    >

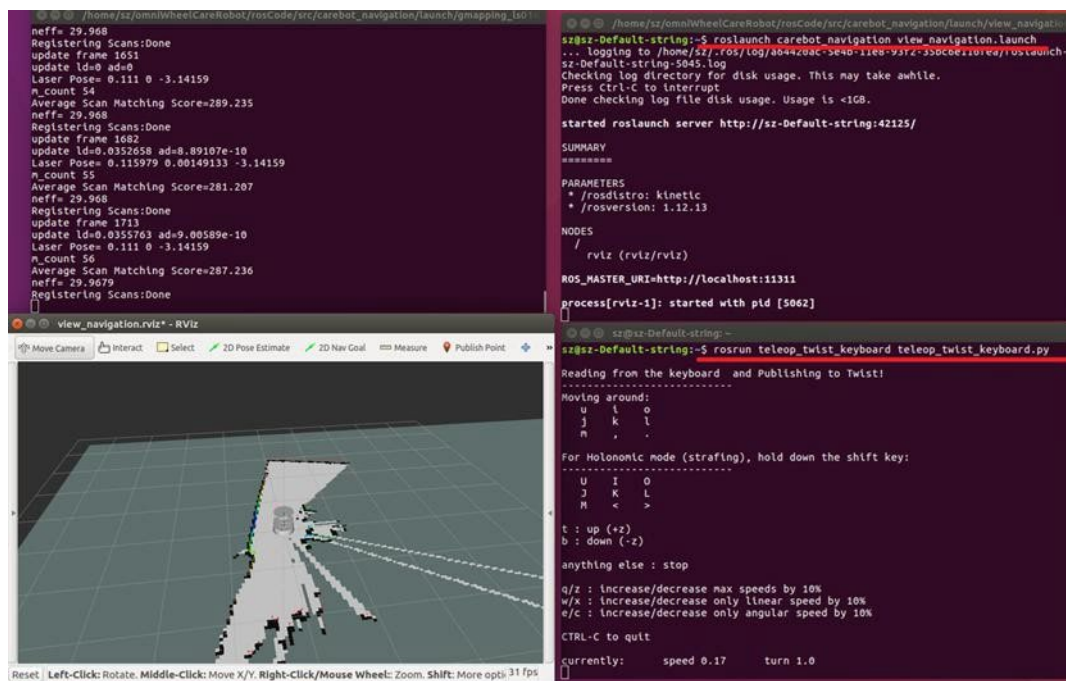
t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit
```

5、在此终端中，通过 i、l、j 等键来控制机器人行走，扫面、



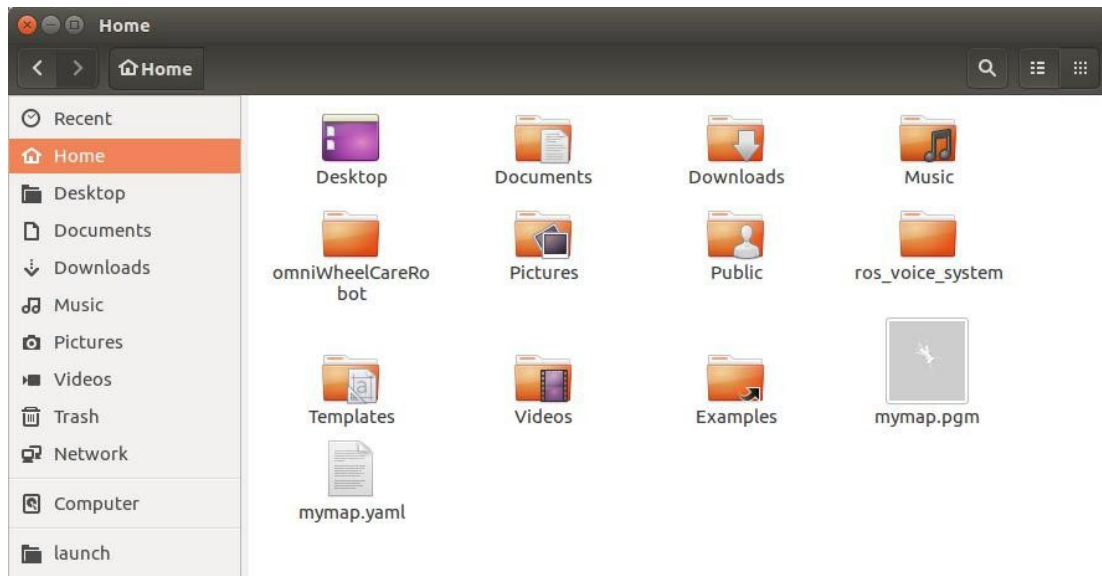
整个空间的地图，完成地图的构建：

6、完成地图的构建后，需要对地图进行保存，此时我们另起终端，通过map_server 命令，保存地图，指令如下：

roslaunch map_server map_saver -f mymap

```
art@art: ~  
art@art:~$ rosrn map_server map_saver -f mymap  
[ INFO] [1565769319.364238897]: Waiting for the map  
[ INFO] [1565769319.616894579]: Received a 672 X 704 map @ 0.050 m/pix  
[ INFO] [1565769319.616931841]: Writing map occupancy data to mymap.pgm  
[ INFO] [1565769319.631892696]: Writing map occupancy data to mymap.yaml  
[ INFO] [1565769319.632033249]: Done
```

7、运行完成后，系统的 home 目录下就会存在两个关于地



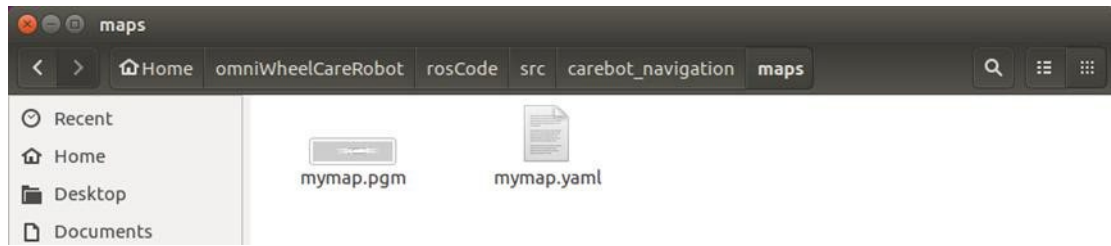
图的文件， 如下所示：

8、因为导航是基于地图实现的，所以我们需要将地图放到指定的路径下面，可以通过MV 指令实现。

```
mv ~/mymap.* omniWheelCareRobot/rosCode/src/carebot_navigation/maps/
```

```
art@art: ~  
art@art:~$ mv ~/mymap.* omniWheelCareRobot/rosCode/src/carebot_navigation/maps/  
art@art:~$
```

9、完成后在 maps 文件夹下，就找到了地图相关文件：



此时，我们即可开始导航的步骤。

注意：执行完成每小节操作后，务必通过 **Ctrl+C** 终结节点，然后关闭终端，随后再去练习下一个小节的内容，否则各个节点之间可能会造成冲突，影响机器人的使用。

任务十一：自主导航

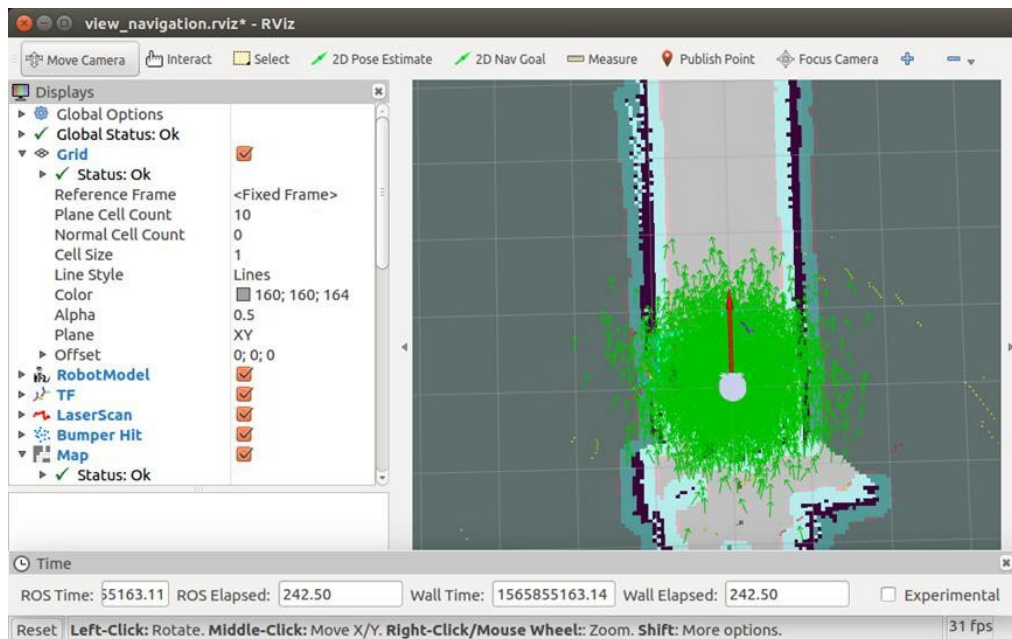
地图创建好之后我们就可以通过算法进行自主导航功能的实现，此处我们使用的是amcl进行自主导航的实现。

1、首先我们需要关闭上一节打开的终端，然后开一个新终端，输入如下命令。

```
roslaunch carebot_navigation ld_navigation.launch
```

同时在桌面上会打开 RViz 界面，加载机器人和地图信息：

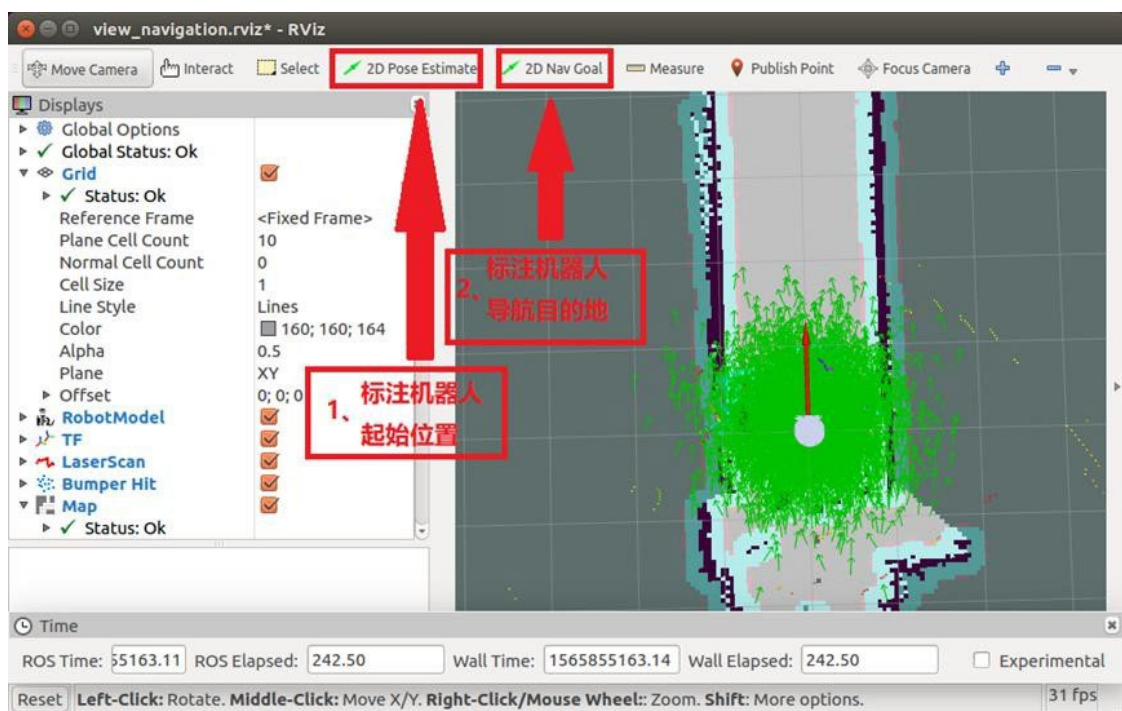
(如果是远程ssh或者vnc连接，取消掉rviz节点，在ubuntu客户端进行打开rviz进行显示)



2、如果地图没有显示出来，则需要打开地图，运行如下指令：

roslaunch carebot_navigation view_navigation.launch

3、点击下图所示红框 1 所指向的按钮，在地图上标注机器人的起始位置和方向；然后点击红框 2 所指向的按钮，在地图上标注机器人要前往的目的地。标注好位置后机器人将自主导航移动到我们所标的目标位置，实现自主导航。



注意：执行完成本节操作后，务必通过 **Ctrl+C** 终结节点，然后关闭终端，随后再去练习下一个小节的内容，否则各个节点之间可能会造成冲突，影响机器人的使用。

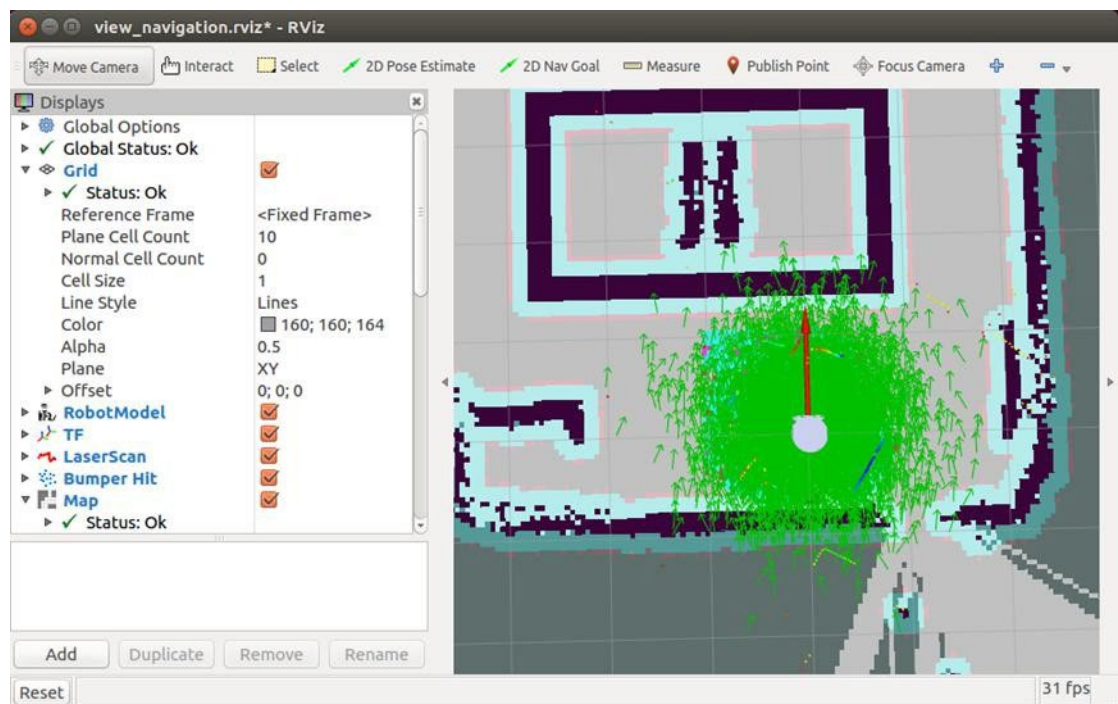
任务十二：多点导航

为了实现机器人更加丰富的功能，此处增加了机器人多点导航的一个效果，可以使机器人按照我们设置好的位置实现一个自主的多点位移效果。

1、首先我们需要关闭上一节打开的终端，然后开一个新终端，输入如下命令，启动导航程序：

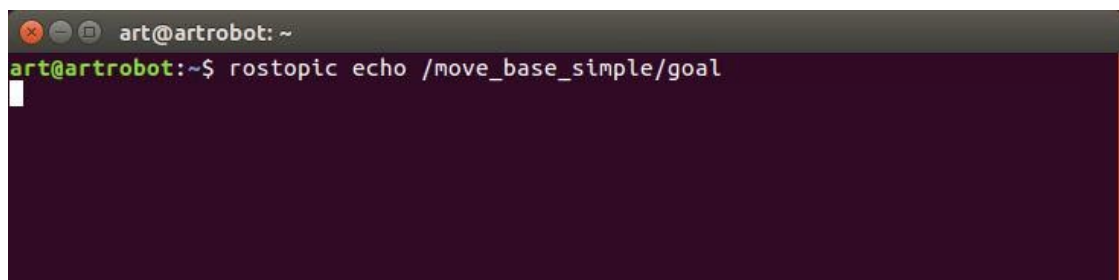
```
roslaunch carebot_navigation ld_navigation.launch
```

同时在桌面上会打开 RViz 界面，加载机器人和地图信息：
(如果是远程ssh或者vnc连接，取消掉rviz节点，在ubuntu客户端进行打开rviz进行显示)



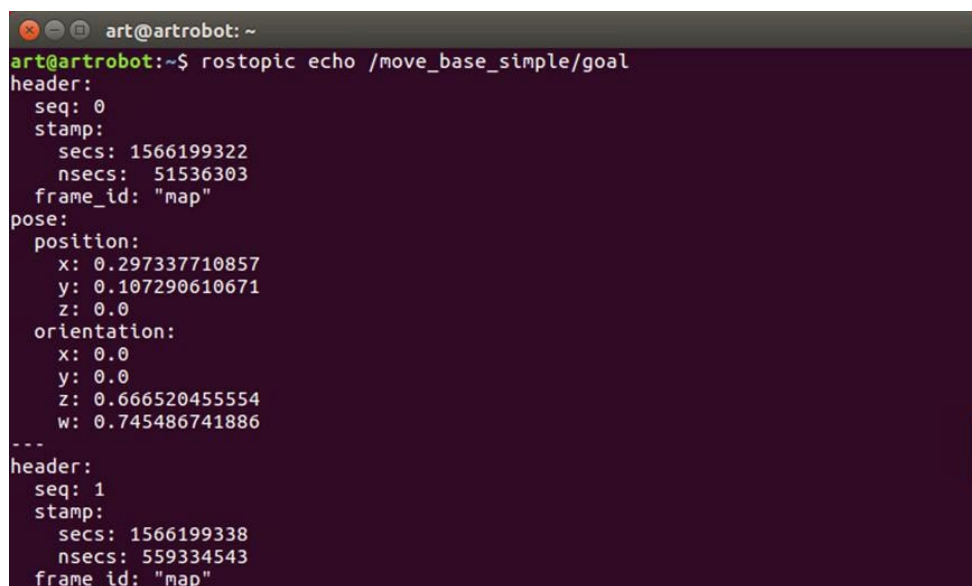
2、打开新终端，输入小车反馈坐标信息的指令，当选定小车的移动位置后将会通过这个终端返回小车的位置坐标以及姿态信息。

rostopic echo /move_base_simple/goal



```
art@artrobot: ~  
art@artrobot:~$ rostopic echo /move_base_simple/goal  
█
```

3、在地图上标注小车的起始位置和方向，然后再标注小车的第一个目标位置，当小车移动到所标第一目标位置后，继续标注小车的第二个目标位置（此时不需要再标注起始位置，而只需再次点击目标位置键设定第二个目标位置），依次标注第三个目标位置、第四个.....

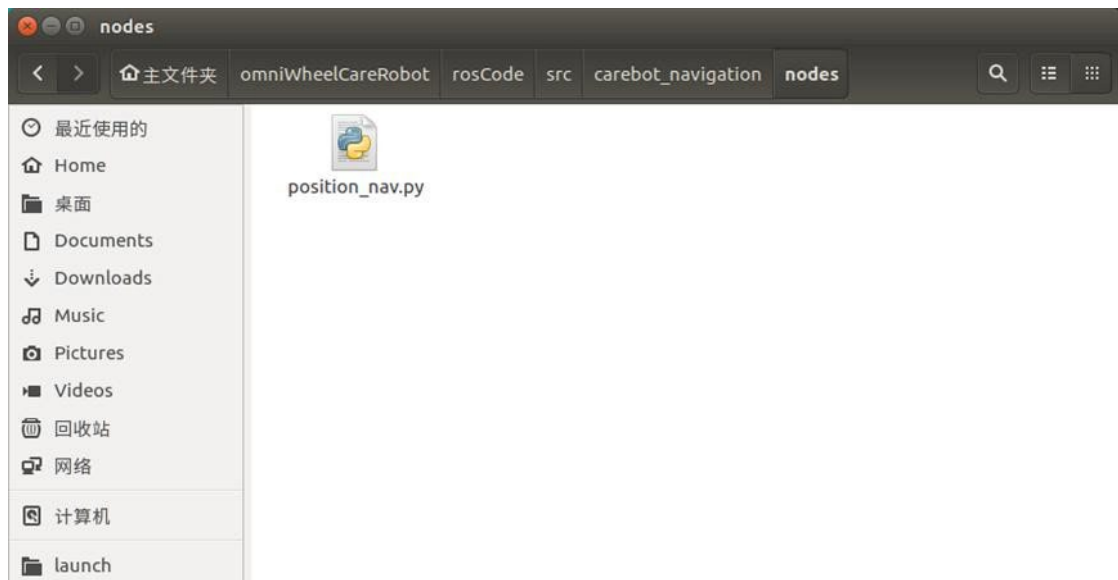


```
art@artrobot: ~  
art@artrobot:~$ rostopic echo /move_base_simple/goal  
header:  
  seq: 0  
  stamp:  
    secs: 1566199322  
    nsecs: 51536303  
  frame_id: "map"  
pose:  
  position:  
    x: 0.297337710857  
    y: 0.107290610671  
    z: 0.0  
  orientation:  
    x: 0.0  
    y: 0.0  
    z: 0.666520455554  
    w: 0.745486741886  
---  
header:  
  seq: 1  
  stamp:  
    secs: 1566199338  
    nsecs: 559334543  
  frame_id: "map"
```

4、在标注小车目标位置的之后，小车每移动到一个目标位置 `rostopic` 终端界面就会相继返回一个位置的坐标点和小车此时的姿态信息。

5、此时我们需要更改此路径下 `position_nav.py` 文件的参数：

文件路径为：`omniWheelCareRobot/rosCode/src/carebot_navigation/nodes`



6、将 rostopic 界面的返回值和下图（来自所打开的文件）进行比对，将文件中的数值改为界面中反馈的值，其中rostopic 界面的x、y、z 和图片中的point 对应，orientation 的 4 位与 Qu

```
# Pose coordinates are then displayed in the terminal
# that was used to launch RViz.
locations = dict()
locations['one'] = Pose(Point(0.582, -0.903, 0.000), Quaternion(0.000, 0.000, -0.630, 0.776))
locations['two'] = Pose(Point(1.281, 1.210, 0.000), Quaternion(0.000, 0.000, 0.643, 0.766))
locations['three'] = Pose(Point(1.579, 1.906, 0.000), Quaternion(0.000, 0.000, 0.897, -0.442))
locations['four'] = Pose(Point(0.618, 0.563, 0.000), Quaternion(0.000, 0.000, -0.032, 0.999))
```

aternion 对应。

7、现在，重启导航步骤：

roslaunch carebot_navigation ld_navigation.launch

```
/home/art/omniWheelCareRobot/rosCode/src/carebot_navigation/launch/amcl_ls01d_lidar.launch
art@art:~$ roslaunch carebot_navigation amcl_ls01d_lidar.launch
... logging to /home/art/.ros/log/2904b61e-bf30-11e9-81e6-68a3c4584893/roslaunch
-art-25799.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://art:45409/

SUMMARY
=====

CLEAR PARAMETERS
* /move_base/

PARAMETERS
* /carebot_amcl/base_frame_id: base_footprint
* /carebot_amcl/global_frame_id: map
* /carebot_amcl/gui_publish_rate: 1.0
* /carebot_amcl/kld_err: 0.1
* /carebot_amcl/kld_z: 0.99
* /carebot_amcl/laser_lambda_short: 0.1
* /carebot_amcl/laser_likelihood_max_dist: 2.0
* /carebot_amcl/laser_max_beams: 300
```

8、同时开启多点导航节点，在 nodes 这个文件夹下面，执行 position_nav.py 文件：

```
art@artrobot: ~/omniWheelCareRobot/rosCode/src/carebot_navigation/nodes
art@artrobot:~/omniWheelCareRobot/rosCode/src/carebot_navigation/nodes$ ./position_nav.py
```

9、此时在地图上只需标注小车的初始位置即可，小车将按照预先设定的位置开始多点导航。

前8个id结尾的参数是无线传感器（开关）的实体id，需要按照具体的设备进行更改，home_ip:为当前智能家居网关的local_ip(访问home_assistant的ip)，该ip出厂已设置好，为192.168.1.190。

token为令牌，需要在智能家居盒子的home_assistant网页上面进行创建。创建过程参考我们提供的《home_assistant模块操作说明.pdf》

我们在实现功能之前，需要将机器人和智能家居网关盒子连接在同一个局域网内。由于我们配备了无线路由器，可以将网关盒子通过网线连接路由器的Lan口，机器人通过无线wifi连接路由器，路由器的wifi热点为以下名称，机器人可以连接路由器的2.4G热点，计算机或笔记本可以通过5G连接。



我们标配的路由器ip为192.168.8.X网段。5G热点的密码为ncut1234，2.4G热点的密码为goodlife。确定机器人的ip地址也为192.168.8.X网段，就可以进行下一步了。

2、在终端输入启动程序：

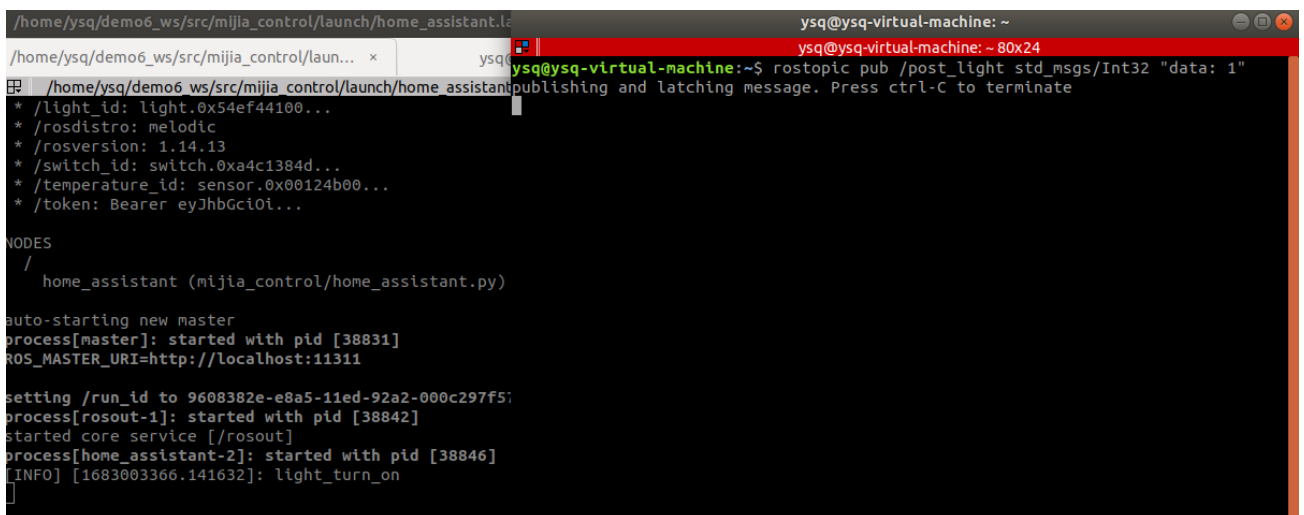
roslaunch mijia_control home_assistant.launch

3、新启一个终端输入：

rostopic list

```
/get_door  
/get_humidity  
/get_temperature  
/post_light  
/post_switch  
/rosout  
/rosout_agg
```

会出现如上图所示话题。话题分为两类。传感器类的话题是“/get_xxxxx”这种类型，我们需要查看什么数据，就需要使用rostopic pub /get_xxxxx的方式进行查询，前面运行launch文件的终端会打印相应的数据，并且通过话题把数据发送出去，以供其他程序使用。执行器类的话题是“/post_xxxx”这种类型，我们需要控制器气开关，就需要使用rostopic pub /post_xxxx的方式控制电气。



```
/home/ysq/demo6_ws/src/mijia_control/launch/home_assistant.l... ysq@ysq-virtual-machine: ~  
ysq@ysq-virtual-machine: ~80x24  
ysq@ysq-virtual-machine: ~$ rostopic pub /post_light std_msgs/Int32 "data: 1"  
publishing and latching message. Press ctrl-C to terminate  
* /light_id: light.0x54ef44100...  
* /rostdistro: melodic  
* /rosversion: 1.14.13  
* /switch_id: switch.0xa4c1384d...  
* /temperature_id: sensor.0x00124b00...  
* /token: Bearer eyJhbGciOi...  
NODES  
/  
home_assistant (mijia_control/home_assistant.py)  
auto-starting new master  
process[master]: started with pid [38831]  
ROS_MASTER_URI=http://localhost:11311  
setting /run_id to 9608382e-e8a5-11ed-92a2-000c297f5:  
process[rosout-1]: started with pid [38842]  
started core service [/rosout]  
process[home_assistant-2]: started with pid [38846]  
[INFO] [1683003366.141632]: light_turn_on
```

（注：如果发布开关或者灯光控制则不能使用Tab键的自动补齐，data: 1-5为开，6以上为关；如果是获取传感器信息的话，需要rostopic echo 对应话题温度为：getTemperature，湿度：getHumidity，等）

如果想与其他节点通信（如语音），也可通过发布对应话题而达到控制效果。

以下使用光照传感器和智能电灯举例：

运行以上第2步的launch文件后，打开一个新的终端，输入指令：

```
rostopic pub /get_illuminance std_msgs/String "data: ""
```

可以采用tab补全的方式完成以上命令。回车后，第2步launch文件的终端会打印：

```
(process[home_assistant-2]: started with pid [3820]  
ros[INFO] [1685120820.373887]: illuminance:176
```

如果输入以下指令：

```
rostopic pub /post_light std_msgs/Int32 "data: 1"
```

可实现控制灯泡电量的功能，如果data后面是6，即关闭灯泡。

注意：执行完成本节操作前，务必已经将传感器加入home_assistant，配置教程请学习配套的服务机器人网关操作手册。

任务十四：综合任务

本节以一个具体家居场景为例，要求机器人自主从房间的入口开始，分别去往3个不同的房间，在不同的房间获取不同的家庭环境数据，或控制相应的电器工作，并使用语音的方式播报相应信息。

首先，需要通过**任务十**的方法构建家居场景地图。其次，采用**任务十二**的方法确定房间内需要停留的多个坐标点，并更新到 `position_nav.py` 文件中。最后，根据**任务十三**的说明准备好智能家居网关以及相应的传感器，做好联网工作。

接下来，将机器人搬运至场地地图的起点，运行以下指令：

1、`roslaunch carebot_navigation ld_navigation.launch`

2、`roslaunch mijia_control assistant.launch`

**3、`cd ./omniWheelCareRobot/rosCode/src/carebot_navigation/nodes/`
`&& python position_nav.py`**

第一条启动了机器人的导航框架。

第二条启动了智能家居网关程序。

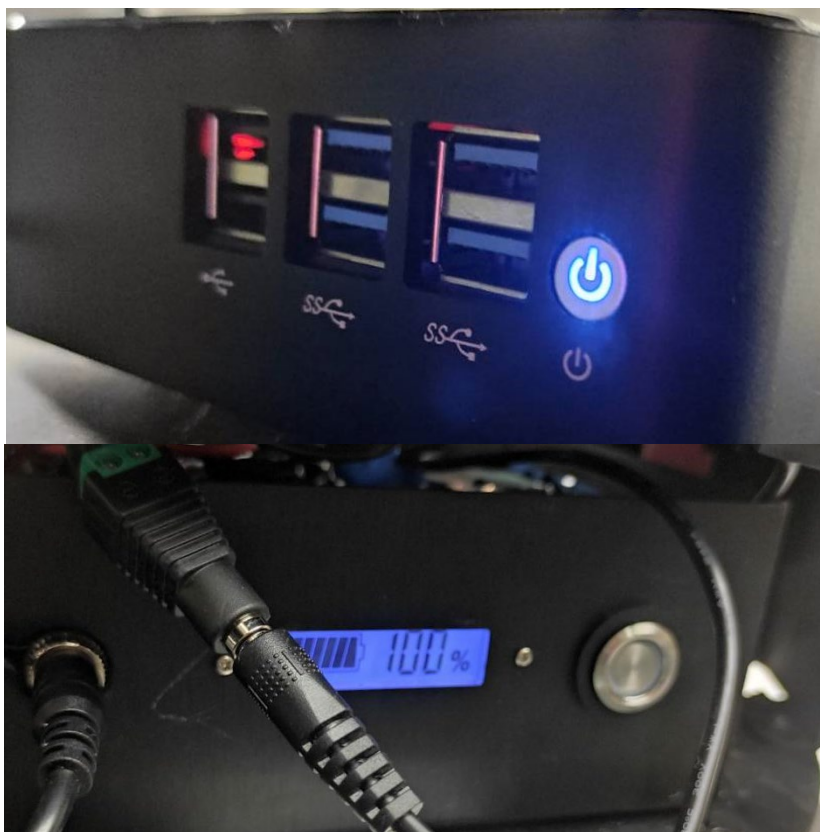
第三条启动了多点导航任务。

当启动第三条以后，机器人便进行多个点的遍历，在相应位置发送话题，获取传感器数据并播报，或者是控制电器工作。

注意事项！！！！

事项一：关闭机器人工控机和底盘时，注意顺序：

1. 先关闭主机ubuntu系统，等主机电源键蓝灯不亮，再关闭底盘开关：



- 2.再拔掉连接底盘和电池的接口，如图所示：



否则工控机所适配的传感器会消耗电池的电量！！！！

！！！！！！！！一定严格按照顺序关闭机器人！！！！！！！！

事项二：显示屏分辨率太小，调整显示屏的分辨率步骤：

[2020-08-26 调整ubuntu 桌面大小 - 简书 \(jianshu.com\)](https://www.jianshu.com/p/2020-08-26)

事项三：系统损坏，用启动盘制作ubuntu镜像步骤：

[\(93条消息\) Ubuntu自定义系统镜像的制作与安装_RobotWoods的博客-CSDN博客](#)